Software developers face many challenges, from unusable application programming interfaces (APIs) [1] to finding relevant code examples to write code without introducing defects [2], which hampers developer productivity. These issues prevent many populations of developers from fully leveraging computational tools, creating barriers to software engineering. For example, end-user programmers (e.g., data scientists or digital artists) often struggle to translate software requirements into working programs [3]. Motivated by these problems, I'm excited to address these barriers and democratize software development during my Ph.D. by synthesizing software engineering (SE), human-computer interaction (HCI), and applied machine learning (ML). I'm interested in studying the various behaviors of different populations of developers to design tools that are tailored to each populations' needs. In my research, I envision using HCI methods to identify problems developers face and SE, HCI, and applied ML to design and evaluate intelligent and personalized tools that address these problems.

After two years of SE internships and leading student developer teams, I experienced the hardships programmers commonly faced and pursued research to understand the SE theory behind these problems. My first SE research experience was a graduate course taught by Dr. René Just. We studied papers in formal and empirical SE research, from abstract interpretation to test generation. I appreciated how formal methods provided rigorous and provably correct tools in SE research, such as in static analysis. Yet, I was drawn to how empirical methods allowed for controlled experimentation on systems and could be easily scaled and iterated upon. Along with a natural language processing graduate student, I researched techniques to improve neural network-based code completion, an emerging technology useful for dynamically typed languages like Python. We explored whether neural network-based code completion was improved by including metadata (e.g., function names and/or comments) during training. After carefully designing an experiment to control for dependent variables, we found that these metadata improved code completion rankings. This experience showed me how applied ML could represent code and thus be a useful tool in my research.

Throughout the SE research course, I realized that even with the important advancements in SE research I was learning about, many were not applied in industry. For example, when I learned about delta debugging, I was impressed by the algorithm's novelty, but was surprised it wasn't widely used even though it was proposed almost twenty years ago. In my opinion, one potential reason for this is because these tools are not designed around developer behavior. I believe this is critical based on my experience in working on a developer tools team at Uber, where we created technical tools by understanding developer needs. For example, by having regular communication channels with Uber developers, they told us that our slow build system prevented code pushes through our CI/CD pipeline. So, I built a Golang metrics pipeline to detect latencies in our build system, where I identified three Golang libraries that contributed to tens of minutes worth of delays that my team could reduce. Since then, I've been passionate about using developer needs to drive the design of SE tools in research.

Now, I'm working with *Professor A* at the intersections of SE and HCI to research how to improve novice developers' problem-solving skills. We're studying explicit programming strategies, human-executable procedures for programming tasks (e.g., binary search for debugging) and have invented a platform to share them. Through interviews and qualitative analysis, our findings indicate that novices use strategies to be more systematic in problem-solving, but often struggle with strategy usability issues, such as unclear wording or defects in the strategy. In using qualitative methods, I realized that they are useful in identifying clear problems that developers face. Yet, the team and I still see opportunities to integrate applied ML into our research, such as mining expert developers' interactions to generate explicit programming strategies for some problem.

Together, my research experiences lead me to believe that SE, HCI, and applied ML may be combined to build intelligent and personalized developer tools. To gain more HCI expertise, I worked in the *Lab B* and researched community-maintained LTE networks for rural populations in developing

nations. I designed tools for people with less technical experience to automate network startup and simplify network management, empowering communities to autonomously manage LTE networks to improve internet access. For additional experience in applied ML, I'm working at the Allen Institute of Artificial Intelligence (AI2). I'm studying why toxic language classifiers classify certain text as toxic. Our initial experiments reveal model biases through ML interpretability methods.

For my Ph.D., I'm excited about ideas in the intersections of SE, HCI, and applied ML. For example, I'm interested in end-user architecture for data scientists. While working with AI2 data scientists, I noticed practicing good SE principles hindered their exploratory programming, as shown in prior work [4]. I'm interested in analyzing the architectural needs of data scientists to create a tool that supports modularity for code reuse to build end-to-end ML pipelines [5], while being flexible enough to support exploratory programming [4]. As another example, I'm also interested in creating API code search tools usable for novice programmers, as they often struggle to find and identify relevant API abstractions [3]. To address the semantic gap between programmers' natural language queries and source code [6], I recently wrote a proposal to mine API code to build a knowledge graph of API domain knowledge based on the API's *software concepts*. *Software concepts* are more understandable to users than API abstractions, as they are ideas that people need to understand to use a piece of software, such as a concept of a *Post* on Twitter [7]. Because of this, I hypothesize that *software concepts* could be a successful basis for code search.

I'm pursuing a PhD to explore my fascination toward SE that I developed first as an SE intern in the industry, next as a student developer lead, and most recently as a SE researcher. After graduation, I will pursue a research career as a postdoc and in the long term, as a professor. I found that I enjoy teaching SE principles to students as a SE teaching assistant, and I'd love to continue teaching during my Ph.D.

CMU is a fantastic place to support my interests during my Ph.D. because of their strong SE and HCI faculty, such as *Professor C* and *Professor D*. *Professor C*'s previous work in SE, HCI, and visualization, as well as *Professor D*'s explorations of API learning and developer tools for data scientists in *Lab E* are a great fit for my passions. This has been confirmed by my conversations with both of these professors, as well as discussing potential collaborations with *Professor D*'s Ph.D. students at the intersections of SE, HCI, and applied ML.

[1] Nam, Daye, et al. "Marble: Mining for boilerplate code to identify API usability problems." *IEEE ASE* (2019).
[2] Head, Andrew, et al. "Interactive extraction of examples from existing code." *ACM CHI* (2018).
[3] Ko, Amy J., et al. "The state of the art in end-user SE." *ACM CSUR* (2011).
[4] Kery, Mary Beth, et al. "Exploring exploratory programming." *IEEE VL/HCC* (2017).
[5] Kim, Miryung, et al. "Data scientists in software teams: State of the art and challenges." *IEEE Transactions on SE* (2017).
[6] Gu, Xiaodong, et al. "Deep code search." *ICSE* (2018).
[7] Jackson, Daniel. "Towards a theory of conceptual design for software." *ACM Onward!* (2015).